[Robot Hackathon] Edge of Intelligence: References

Viam x Intel hackathon, Nov 13 & 14, 2026

Your station

Your station is a machine in the **hackathons** organization of the Viam app, named on the label of its compute box (`armfarm` plus a number). Here's what's configured for you:

Resource	Model	What it is	What you do with it
`arm`	`viam:ufactory:xArm6`	The follower arm. Runs at 60 deg/s for its own moves	Move it from its test card, or let teleop and your policy drive it
`gripper`	`viam:ufactory:gripper`	The two-finger gripper on the arm	Squeeze the leader's gripper to close it under teleop. Your policy sets it
`cam`	`viam:orbbec:astra2` or `viam:camera:realsense`	The wrist camera, color and depth	Record it in every episode; it is what your policy sees
`overview-cam`	`rdk:builtin:webcam`	The scene camera on the stand, 640 by 480	Record it in every episode
`leader-arm`, `leader-gripper`	`devrel:so101:arm`, `devrel:so101:gripper`	The SO-101 leader arm and its gripper	Control it to teleoperate the arm while recording.
`teleop`	`viam:teleop:leader-follower` 0.0.2	Maps the leader's end-effector pose onto the arm through the motion service	Start and stop it
`capture-control`	`viam:capture-control:capture-control-sensor` 0.1.1	Says what to record and when: `arm` joint positions, `gripper` state, `cam` and `overview-cam` images	Leave it to the webapp. Nothing is recorded while it is idle
`data_manager`	the built-in data manager	Records what the sensor asks for and syncs to the Viam app every 6 seconds	Watch its uploads on the DATA tab
`capture-webapp`	`viam-labs:capture-control:webapp` 0.2.0	Serves the capture control webapp from the station on port 8888	Record episodes and review them
`home-pose`, `leader-home`	`erh:vmodutils:arm-position-saver` 0.4.7	Saved poses for the follower and the leader	Press **H** in the webapp to send every saver in the chosen home set
`table`, `ceiling`, walls	`erh:vmodutils:obstacle`	What the motion service plans around	Leave them
`discovery-cam`, `discovery-arm`, `leader-calibration`		Setup helpers; `leader-calibration` is disabled	Leave them

Before the arm moves

- **The E-stop is the red button on the xArm controller.** Pressing it immediately stops the arm. You'll need to release the button to move the arm again.
- **Speed caps apply to every move.** Teleop caps the follower at 250 mm/s and 180 deg/s. The arm's own moves, including going home, run at 60 deg/s. A policy run is limited by the controller's per-tick joint clamps.

Teleop

Teleop mirrors the leader's end-effector pose onto the arm through the motion service, which works out the arm's joint angles for each pose. That is how a five-joint leader drives a six-joint arm.

Do	Where	How
Start or stop	The capture control webapp	Press **T**. While recording, **T** stops teleop and ends the episode
Start or stop	`teleop` service's DoCommand card	DoCommand `{"command": "start"}` or `{"command": "stop"}`
Check	`teleop` service's DoCommand card	DoCommand `{"command": "status"}`

On start, the leader moves itself to the pose that matches the arm, then mirroring begins. Take hold of the leader only when it has stopped.

If the arm is not moving, send `{"command": "status"}` and read `pipeline_error`: it names what stopped teleop (an arm fault, the E-stop, a planning failure). Clear the cause, then start again. Every status field is in the teleop module's README.

Move the leader slowly. The speed caps (250 mm/s, 180 deg/s) are why the arm may lag when the leader moves too fast.

If the leader arm needs recalibrating

Enable the `leader-calibration` sensor on the CONFIGURE tab (it is on the station, disabled) and open the SO-101 setup application. Pick your machine and `leader-calibration`, then choose **Calibration Only**. The pose you hold at the homing step becomes 0 on every joint, so hold the zero pose there, then move every joint through its full range when asked. Afterwards restart the `so101-arm` module from its card, since the arm reads its calibration when it starts, and disable `leader-calibration` again. The workflow's commands and states are in the so101-arm module's README.

Recording episodes

An episode is one demonstration, start to stop. In Viam it is one **sequence**, tagged with at least `task:<task>`, `session:<id>` and `fps:<fps>`, holding `arm` joint positions, `gripper` state and both cameras' frames at the session's frame rate.

Open the webapp at https://capture-control-webapp_viam-labs.viamapplications.com. Under **Robot setup**, pick your machine, confirm the `teleop` service is listed, choose the home set (`home-pose` and/or `leader-home`), and type your task names.

Key	Does
Enter	Starts a session
Space	Starts the first episode; after that, ends the episode and starts the next

Key	Does
Esc	Stops the session
1 to 9	Picks the task for the next episode
T	Toggles teleop. While recording, stops teleop and ends the episode
H	Homes every saver in the home set. Not while recording
?	Shows the shortcut overlay

Where it appears: the Viam app, **DATA** tab, **SEQUENCES**. Each row shows the time range, the machine part, the resources recorded and the tags. Data syncs every 6 seconds; the first entries can take 30 to 60 seconds to show up.

Review: Review session after a session ends, or **Episodes** for older ones. **K** tags the episode `review:keep`, **D** tags it `review:discard`. Nothing is deleted from review.

Keep the session FPS at 10 and type the task exactly. Training reads the task tag as the instruction and rejects sequences recorded at mixed rates.

Track 1: Teach it by showing

Train a policy from your own demonstrations and run it on the station through Viam. The packing task is a ladder, tiers 1a to 5, attempted in any order in a ten-minute judged run; fewest demonstrations recorded breaks ties, and the shipped dataset counts toward your total.

Tier	The robot	Points
1a	Picks one item from a fixed position and puts it in the box	5
1b	Same, from a randomized position	5
1c	Same, with distractor items present	5
1d	Two items in sequence	5
2	Packs the three items on the order and leaves the two that are not	10 per correct item, minus 10 per off-order item packed
3	Fits one awkward item from a menu: a bottle laid flat, a bag, a deformable item	20
4	Closes the lid	20; 10 for moving the lid without closing it
5	Recovers when a judge moves an item or adds a distractor mid-episode, from a fixed list of scripted interventions	20

Your starting point

What	Where	Notes
The kit dataset: tier 1 demonstrations recorded with the same leader arm, cameras and mapping as your station	hackathons org, DATA, DATASETS	`swag-packer-baseline`, id `6af63a56d97a66694a1e25a7`
Policy module `viam-labs:openvino-physicalai`	Viam registry, 0.5.0	Model `viam-labs:openvino-physicalai:policy`, `linux/amd64` only. Its README has every attribute, training argument and measured latency
Controller module `viam-labs:vla`	Viam registry, 0.3.0	Model `viam-labs:vla:controller`. Its README has every config field and status field
The training script `physical-ai-studio` and its container	Script public in the viam-dev org; container registered in the hackathons org	Script `e76d1b3b-0468-4efd-bb7f-fb1d2b352fcb:physical-ai-studio` version `9`; container `physicalai-train 0.3.0, bimanual + grippers`, id `1045eccc-b3b7-48fe-86cf-5230b82c336d`

No ACT or SmolVLA policy ships with the station. The first policy you run is the one your first training job produces.

Setup

1. **Add the policy module** `viam-labs:openvino-physicalai`. Its first run creates a Python 3.12 environment under the module's data directory and installs `physicalai` 0.3.0, which can take several minutes.
2. **Reference your model package** in `packages`: `{"name": "<model>", "package": "6d6c7293-bc67-43e4-9c8b-31c76aac27d3/<model>", "type": "ml_model", "version": "<version>"}`, the `--model-name` and `--model-version` of a finished training job. The package's `viam.json` lists the cameras, arm, gripper and `tasks` the policy was trained with.

3. **Add the policy service**, name `policy`, model `viam-labs:openvino-physicalai:policy`, with `model_path` `${packages.ml_model.<model>}` and `device`. ACT runs on `NPU`, `GPU` or `CPU`; SmolVLA runs on `GPU` and refuses `NPU`.
4. **Wait for ready.** Loading runs in the background. On the test card send `{"command": "status"}` until it returns `{"state": "ready", "error": ""}`. Then `{"command": "specs"}` shows what the policy expects, including its `image_feature_keys`.
5. **Add the controller module** `viam-labs:vla` and a service `controller`, model `viam-labs:vla:controller`:

```
{
  "policy_service": "policy",
  "arm": "arm",
  "cameras": {"images.cam": "cam", "images.overview_cam": "overview-cam"},
  "state_joint_indices": [0, 1, 2, 3, 4, 5],
  "gripper": {"type": "do_command", "name": "gripper", "read_key": "pos", "open_value": 840, "closed_va
lue": 0},
  "task": "put the black hat in the cardboard box",
  "fps": 10.0,
  "safety": {"max_vel_degs_per_sec": 60.0, "max_start_delta_degs": 15.0, "stop_on_error": true}
}
```

`cameras` must cover every key in `specs.image_feature_keys`. The gripper block's values are what the station gripper's `DoCommand` returns: `pos`, 840 open and 0 closed, the same values the kit dataset was recorded with.

1. **Before a run:** stop teleop and move the leader arm out of the workspace. `start` refuses if the policy's first action is more than 15 degrees from where the arm is.

The loop

Step	How
Record	The shared Recording section. Leave FPS at 10. The `task:` tag is the instruction
Build a dataset	DATA tab, DATASETS, **Create dataset**, Data type **Sequence Data**. Open it, **Add Data**, select the sequences tagged `review:keep`, **Add**. The sidebar shows the sequence count
Train	The command below. `viam train logs --job-id=<id>` streams progress with a minutes-left estimate
Deploy	Bump the package `version` in `packages` to the new model version and save. The policy reloads in the background; `status` returns `ready` when it is loaded
Run	On `controller`, `{"command": "start"}`, or with `"task"` to override the configured one. It returns at once; `status` then shows `running` and `measured_fps`
Stop	`{"command": "stop"}`. The arm stays where it is
Retrain	A new dataset with all kept episodes, a new `--model-version`, and the new version in `packages`

The training command, run in the hackathons organization:

```
viam train submit custom from-registry --org-id=6d6c7293-bc67-43e4-9c8b-31c76aac27d3 --dataset-id=<data
set id> \
  --model-name=<your model> --model-version=1 \
  --script-name=e76d1b3b-0468-4efd-bb7f-fb1d2b352fcb:physical-ai-studio --version=9 --container-id=1045
eccc-b3b7-48fe-86cf-5230b82c336d \
  --args=policy=<act, smolvla or pi05>,arm=arm,grippers=gripper,gripper_key=pos,gripper_open=840,camera
s="'overview-cam cam'",fps=10,image_size=224
```

`policy=smolvla` fine-tunes SmolVLA instead of training ACT. The script reads the instruction from a `cmd:` or `task:` tag on each sequence, so the webapp's `task:` tags are read; `task=` covers sequences without one. Every `--model-version` must be new. The other arguments and their defaults are in the physicalai module's README.

Pre-warm list

Start it early	Why	How long
The policy module's first run	Creates the Python environment and installs `physicalai`	Several minutes
A model's first load on your device	Compiles the model for the NPU or GPU in the background; `status` is `loading` until it ends	A few minutes for ACT, longer for SmolVLA
Your first training job, on the kit dataset	The first policy you can run; your own demonstrations go into the next one	Roughly an hour or more on a cloud GPU, plus a 1 GB checkpoint download for SmolVLA

Things to keep in mind

- **The gripper stalls the arm.** The arm freezes for up to 10 seconds at every gripper change during a policy run, because the ufactory gripper's `set` call blocks until the jaw settles. No configuration avoids it, and the fix in viam-modules/viam-ufactory-xarm#145 is not merged and not on the station image.
- **Chunks are blind.** ACT predicts 100 actions per inference, about 10 seconds with no new camera input at 10 fps; `n_action_steps` at training time sets how often it replans. SmolVLA's chunk is fixed at 50.

Track 2: Big models, small box

Run pi0.5, about 3 billion parameters, on the station's Arc GPU and have it do the packing task from language. Judged on tiers 1 to 6, orders given in language. Each team may submit at most one additional fine-tune, before the cutoff at 9 pm on day one; a pi0.5 fine-tune runs 4 to 6 hours on average.

Tier	The robot	Points
1a to 5	The Track 1 ladder, from a language order	As in Track 1
6a	Packs the item named in the order	10
6b	Picks by attribute ("the red one")	10
6c	Picks by count ("two of the small boxes")	10
6d	Follows a conditional or negated order ("everything except the fragile one")	20

Everything in Track 1 applies: the same modules, the same controller, the same loop. This section lists only what differs.

Your starting point

What	Where	Notes
A pi0.5 checkpoint pre-fine-tuned on the xArm6, the kit and the box, with language labels, exported for OpenVINO	hackathons org ML model package	`swag-packer-baseline`; its training job ran 3 h 37 min. Its `viam.json` lists the trained instructions, among them `put the black hat in the cardboard box`
The kit dataset with language labels	hackathons org DATA	`swag-packer-baseline`, id `6af63a56d97a66694a1e25a7`, the same dataset as Track 1

Setup differences

1. **Device is `GPU`.** pi0.5 does not compile for the NPU; the policy refuses `NPU` at load and reports it in `status.error`. `CPU` works but takes about 11 seconds per chunk.
2. **The package is large.** A pi0.5 model version is about 6 GB. The download and the first GPU compile happen in the background after you save; poll `status`. If it reports `failed` with a load timeout, raise `load_timeout_s` above 600.
3. **The instruction goes with each run.** Configure a default `task` on the controller, then override it per run with `{"command": "start", "task": "<order>"}`. The package's `viam.json` lists the `tasks` the checkpoint was trained on.
4. **Chunks are fixed at 50** by the checkpoint. Set `n_action_steps` at training time to replan more often; at 230 ms per chunk on the GPU, 50 actions at 10 fps is 5 seconds of motion per prediction.

Fine-tuning

The same training command as Track 1 with `policy=pi05`. The job fine-tunes `lerobot/pi05_base` with LoRA, downloads a 15 GB checkpoint first, runs 4 to 6 hours on average on a datacenter GPU, and exports about 6 GB. That is one cycle, and one is all the weekend allows. The pi0.5 arguments (`lora_rank`, `num_steps`, `n_action_steps`) are in the physicalai module's README.

Pre-warm list

Start it early	Why	How long
The baseline's download and first GPU compile	6 GB to fetch, then a compile in the background	Expect 10 minutes or more; the first NPU compile of pi0.5 on the development NUC took 6.5 minutes
Your dataset and the one fine-tune submission	One cycle is all the weekend allows	4 to 6 hours on average; the cutoff is 9 pm on day one

Things to keep in mind

- **Recovery is yours to build.** The controller halts on a failed tick when `stop_on_error` is true.
- **The gripper stall and the safety clamps** from Track 1 apply unchanged.
- **MolmoAct 2** has no OpenVINO conversion anyone knows of. The first team to run it on the Arc GPU through Viam takes the named prize.

Track 3: Spot it, sort it

Defect detection and response: cameras and an anomaly model find defective items, something decides what to do with each, and the arm sorts them into the box, the reject bin or the hold area. Scored on the judge set: detection precision and recall from your own log (30%), items ending in the right place (30%), latency from an item entering view to the arm starting its response (10%), device placement with a measured reason, shown through diagnostics (10%), and a five-minute design explanation (20%). The judges include a defect type you have not seen.

Your starting point

What	Where	Notes
OpenVINO runtime module `viam:viam-openvino`	Viam registry, 0.0.6	Models `mlmodel`, `yolo`, `rf-detr`, `diagnostics`. The namespace moved from `erh` to `viam` on 2026-11-02; READMEs that say `erh:openvino` mean this module
Anomaly module `viam-labs:anomalib`	Viam registry, 0.1.5	Model `viam-labs:anomalib:vision`, plus the training script. 0.1.5 fixes the vision service's test card; on 0.1.4 the service works through the API but the test card errors
Reference anomaly package `fastener-patchcore` **version 1**	hackathons org ML models	PatchCore trained on 169 good and 83 marked-fastener frames from one station's wrist camera, 256 px, OpenVINO (`model.xml`); it learned that station's camera pose, lighting and parts
Dataset `fastener-anomaly`, **id** `6af5edb82deac4224da1b4cd`	hackathons org DATA	The frames behind `fastener-patchcore`: 169 `normal` and 83 `anomalous`

Setup

1. **Add diagnostics, which brings the runtime with it.** On the CONFIGURE tab, add the `viam-openvino/diagnostics` configuration block (type GENERIC) and name it `ov-diag`. That also adds the `viam:viam-openvino` module. On the `ov-diag` test card send `{"command": "list_devices"}` and expect `CPU`, `GPU` and `NPU` entries; on a station with drivers present the module's first run took about 4 seconds. If a device is missing, the driver checks are in the viam-openvino README. The OpenVINO pieces are services, not components: the module appears in the modules section and `ov-diag` under services.
2. **Add the anomaly chain.** Train a model (next heading) or start from the `fastener-patchcore` package. In `packages`, reference the model: `{"name": "fastener-patchcore", "package": "6d6c7293-bc67-43e4-9c8b-31c76aac27d3/fastener-patchcore", "version": "1", "type": "ml_model"}`. An mlmodel service `ov-anomaly`, model `viam:openvino:mlmodel`, with `model_path` `${packages.ml_model.<your model>}/model.xml` (or `model.onnx`), `device` `GPU` and `inference_precision` `f32`. Then add the module `viam-labs:anomalib` and a vision service `anomaly`, model `viam-labs:anomalib:vision`, with `mlmodel_name` `ov-anomaly` and `camera_name` `cam`. `threshold` is yours to choose; the module default is 0.2. The service answers only for its configured `camera_name`. To see its boxes on the CONTROL tab, add a `transform` camera with `source` `cam` and a `detections` pipeline step whose `detector_name` is `anomaly`; each such camera runs the model on every frame it shows.
3. **Add one diagnostics service per model you want to measure**, each with `mlmodel_name` set to that mlmodel service. A `mlmodel_name` key in the command overrides it per call.
4. **The decision and the motion** are yours: the motion service with a 3D position from the depth camera, a policy trained as in Track 1, or a vision-language model through the `viam:openvino-genai` module, whose `vlm` block is in Configuration Blocks.

Train an anomaly model

1. Capture images of good items, either through `capture-control`, which switches `cam`'s data capture on the station, or with a `transform` camera with `source` `cam`, which has its own data capture settings and is not gated by `capture-control`. On the DATA tab, add at least 10 images of good items to a dataset and tag them `normal`. Optionally tag 4 or more defective items `anomalous`; fewer than 4 are ignored, and without them the job evaluates against synthetic anomalies.
2. Submit the job. The script and container are registered in the hackathons organization:

```
viam train submit custom from-registry --org-id=6d6c7293-bc67-43e4-9c8b-31c76aac27d3 --dataset-id=<data
set id> \
  --model-name=<your model> --model-version=1 \
  --script-name=hackathons:anomalib-train --version=0.4.0 \
  --container-id=9accd6e7-2992-47c9-9662-2ca5298fa1f1 \
  --args=model=patchcore,image_size=256
```

1. `viam train logs --job-id=<id>` shows the image AUROC and F1 at the end, and the package's `metadata.json` keeps them after the logs expire. A training job typically takes about 10 minutes. Every `--model-version` must be new; to retrain, increase it.
2. The output is a registry ML model package (`model.xml` and `model.bin`). For ONNX use `--script-name=hackathons:anomalib-train-onnx`, add `export_format=onnx` to `--args`, and give the model a new name; a model keeps the framework of the script that trained it.

`patchcore` is the default. The other models that work, the ones that do not, every argument and its default are in the anomalib service README.

Pre-warm list

Start it early	Why	How long
The first start of each model on its device	OpenVINO compiles the model for the device once and caches it; under `AUTO` the service answers from the CPU while the accelerator compiles	Seconds for PatchCore on the GPU; minutes for a large model
`list_devices` on `ov-diag`	Lists the devices the runtime can use	Seconds
A first training job	10 `normal` images are enough to submit one	About 10 minutes

Things to keep in mind

- **PatchCore does not compile on the station's NPU.** The NPU compiler reports "Node 0 doesn't respect Custom Operator format". Bare `AUTO` tries NPU, then GPU, then CPU, and falls back after a warning; an explicit `NPU` never starts, and an explicit `GPU` that is missing is a hard failure. Measured on `armfarm04`: 23 ms per image on the GPU, 75 ms on the CPU.
- **PatchCore on the GPU needs `inference_precision` `f32`.** With the GPU's default f16, `pred_score` is right but `anomaly_map` comes back all zeros, so no boxes and an empty heatmap.
- **The NPU needs static input shapes.** No `resize` in `preprocess` on an NPU model; set `input_shape` for a model exported with dynamic dimensions.
- Thresholds, `num_requests`, preprocessing and the cache are in the viam-openvino README and the anomalib README.

Using AI

- **The AI assistant in the Viam app**, on your machine's page.
- **Your own AI agent.** Point it at the Viam docs, the module READMEs, and the agent-skills repository: Claude Code skills for the Viam SDKs, the CLI, the ML pipeline and live machines. The docs page Use Viam from an AI agent covers the CLI, SDK and MCP routes.
- **The Viam MCP server** at https://app.viam.com/mcp. Add it to Claude, Claude Code, ChatGPT or Codex as a remote MCP server; it signs you in to Viam with OAuth and works within what your account can already access. Its tools read and edit machine configuration and call APIs on live machines, so an agent can move the arm. In Claude: Customize, Connectors, Add. In ChatGPT: Plugins, then the + icon, with developer mode enabled under Settings, Security and login. The tool list is in the MCP server reference.

Modules worth a look

The Viam registry is where to look for a module for anything the station does not have yet. These general-purpose ones fit a sort, pick or place pipeline on the station and are worth a look. Each page has the README, the configuration attributes and the versions.

Module	What it does
`viam-labs:grasp-pose`	A pose tracker that turns a vision service's 3D detections into ranked top-down grasp poses for a parallel-jaw or suction gripper, plus each object's fitted box to use as an obstacle in a motion plan. Needs a vision service that serves 3D detections, such as the two below
`viam:detections-to-segments`	A vision service that turns a 2D detector's boxes plus the depth image into 3D segments
`viam:obstacles-pointcloud`	Vision services that find well-separated objects above a flat plane from a point cloud or a depth image, with no detector
`viam:apriltag`	AprilTag pose tracker and detector, for a known pose on a tag
`viam:camera-calibration`	ChArUco board pose tracker for calibrating a camera against the arm